Dreamer V2.

Mastering Atari with Discrete WM

RSSM {
- Recurrent Model: $h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1})$.
- Representation Mode: $z_t \sim q_\phi(z_t | h_t, x_t)$.
- Transition Predictor: $\hat{z}_t \sim p_\phi(\hat{z}_t | h_t)$.
- Image Predictor: $\hat{x}_t \sim p_\phi(\hat{x}_t | h_t, z_t)$
- Reward Predictor: $\hat{r}_t \sim p_\phi(\hat{r}_t | h_t, z_t)$
- Discount Predictor: $\hat{\gamma}_t \sim p_\phi(\hat{\gamma}_t | h_t, z_t)$.

Loss function for learning WM →

$$\mathcal{L}(\phi) = \mathbb{E}_{q_\phi(z_{1:T} | a_{1:T}, x_{1:T})} \Big[\sum_{t=1}^{T} \underbrace{-\ln p_\phi(x_t | h_t, z_t)}_{\text{image log loss}} \underbrace{- \ln p_\phi(r_t | h_t, z_t)}_{\text{Reward log loss}} \underbrace{- \ln p_\phi(\gamma_t | h_t, z_t)}_{\text{Discount log loss.}} + \underbrace{\beta KL[q_\phi(z_t | h_t, x_t) \,||\, p_\phi(z_t | h_t)]}_{\text{KL loss.}} \Big]$$

Actor → $\hat{a} \sim p_\psi(\hat{a}_t | \hat{z}_t)$

Critic →

$$v_\xi(\hat{z}_t) \approx \mathbb{E}_{p_\phi, p_\psi}\Big[\sum_{\tau \geq t} \hat{\gamma}^{\tau - t} r_\tau \Big]$$

Critic Loss fn →

$$V_t^{\hat{\lambda}} = \hat{r}_t + \hat{\gamma}_t \begin{cases} (1-\lambda)\, v_\xi(\hat{z}_{t+1}) + \lambda V_{t+1}^{\hat{\lambda}} & \text{if } t < H \\ v_\xi(\hat{z}_H) & t = H. \end{cases}$$

$$\mathcal{L}(\xi) = \mathbb{E}_{p_\phi, p_\psi}\left[\sum_{t=1}^{H-1} \frac{1}{2}\left(v_\xi(\hat{z}_t) - sg(V_t^{\hat{\lambda}}) \right)^2 \right]$$

Actor Loss Function →

$$\mathcal{L}(\psi) = \mathbb{E}_{p_\phi, p_\psi}\left[\sum_{t=1}^{H-1} \Big(\underbrace{-\rho \ln p_\psi(\hat{a}_t | \hat{z}_t) sg\left(V_t^{\hat{\lambda}} - v_\xi(\hat{z}_t)\right)}_{\text{Reinforce}} - \underbrace{(1-\rho) V_t^{\hat{\lambda}}}_{\text{Dynamics Backprop.}} - \underbrace{\eta H[a_t | \hat{z}_t]}_{\text{Entropy Regularizer}} \Big) \right]$$

Algo 1 - Straight-Through Gradients with Automatic Differentiation.

Sample = one_hot (draw (logits)) (No Gradients)

probs = Softmax (logits) (Want Gradient of this)

Sample = Sample + probs - stop_grad(probs). (Has Grad of Probs)

(This Algo effects 2 changes. (i) the Sampling One
(ii) then for the Gradient Flow.

$$\frac{d(\text{sample})}{d(\text{logits})} = \frac{d(\text{probs})}{d(\text{logits})}$$

The Gradient flow only through the probs term, exactly as if sample were probs.

If sample = hard, then hard came from .sample()

$$\frac{d(\text{sample})}{d(\text{logits})} = 0.$$

KL Balancing →

Reason for KL Balancing to be use instead of only KL :-

In V1, KL pulls both sides simultaneously. The prior is dragged towards posterior, but the posterior is also dragged toward the prior, because the gradients flow through both. (Symmetric pulling)

Ex → (Net Practice) - The Coach dumbs down to match the student

Now, KL Balancing (Asymmetric Pulling)

V2 fixes this by splitting the KL into 2 pieces with a stop-gradient on each side -

$$\mathcal{L}_{kl} = \underbrace{\alpha \cdot KL(sg(\text{posterior}) \,||\, \text{prior})}_{\text{Posterior is frozen, Only prior gets gradient (Training prior to match posterior)}} + \underbrace{(1-\alpha)\, KL(\text{posterior} \,||\, sg(\text{prior}))}_{\text{Prior is frozen Only the posterior gets gradient}}$$

Ex = $\alpha = 0.8$ $\quad 1-\alpha = 0.2$

So, 0.8 v/s 0.2 split. The prior is being trained 4x harder to match the posterior than vice versa.

Ex → (Net Pratice) - The Coach now teaches the student finely, and is only allowed to lower the bar a tiny bit if the student is completely failing. Mostly, the student rises to the coach's level.

Algo 2 : KL Balancing with automatic Diff ~ -

```
KL_loss = alpha * compute_kl(sg(approx. posterior), prior)
    + (1 - alpha) * compute_kl(approx. posterior, sg(prior))
```

Analytic Grad-
/ Pathwise / Reparameteriz^n gradients).

The precision properties :-
(i) Bias - 0 The ∇ you compute is the exact ∇ of the loss. No approx. anywhere
(ii) Variance - Low Each sample gives a near - deterministic gradient. The randomness is isolated in ϵ, which doesn't depend on θ.

This is the Combination of Zero Bias & Low Variance (Gold standard)
If you can use analytic gradients, you should.

REINFORCE →

Reward Increment = Non-negative Factor × off Reinforcement × Characteristic Eligibility.

{Ronald Williams}
(1992)

We want, $\nabla_\theta \mathbb{E}_{a\sim\pi_\theta}[R(a)]$

$$\mathbb{E}[R(a)] = \int \pi_\theta(a) \cdot R(a)\, da.$$

Diff^n w.r.t θ.

$$\nabla_\theta \mathbb{E}[R(a)] = \int \nabla_\theta \pi_\theta(a) \cdot R(a)\, da.$$

$$= \int \pi_\theta(a) \frac{\nabla_\theta \pi_\theta(a)}{\pi_\theta(a)} \cdot R(a) \cdot da$$

{$\pi_\theta(a)$ alone & below Multiply}

$$\int \pi_\theta(a) \nabla_\theta \log \pi_\theta(a) \cdot R(a) \cdot da.$$

$$\mathbb{E}_{a\sim\pi(a)}[\nabla_\theta \log \pi_\theta(a) \cdot R(a)]$$

Last line was Policy Gradient Theorem -

$$\nabla_\theta J(\theta) = \mathbb{E}\left[R \cdot \underbrace{\nabla_\theta \log \pi_\theta(a|s)}_{\text{is the direction in parameter space that would make this particular Act}^{n} \text{ more likely. If } R \text{ is +ve (Good Act}^{n}\text{) nudge } \theta \text{ that way. If } R \text{ is -ve, nudge the opposite way}} \right]$$

"Do more of what worked, weighted by how well it worked."

What REINFORCE needs -

1. Ability to sample an action
2. Ability to compute $\log \pi(a|s)$
3. A scalar reward R.

Actor - Critic →

The 2 Players (both live inside the dream.

Actor $(h, s) \to \pi(a|s)$: decides. "what do I do here ?" = Batsman

Critic $(h, s) \to V(s)$: "How good is this state ?" = Coach.

Core Eq^n.

Return → $G_t \Rightarrow \sum \gamma^k r_{t+k}$

Value → $V(s) = \mathbb{E}[G_t | S_t = s]$

Pol. Grad → $\nabla J = \mathbb{E}[\nabla \log \pi(a|s) . G_t]$

Baseline := Advantage (why Critic exists)

$A_t = G_t - V(S_t)$

$A > 0$ do More

$A < 0$ do Less

Now λ - Returns (How we estimate the target G_t)

Tension : MC = Unbiased, High Var

1-step TD = Biased, Low Var

n-step : n real rewards + bootstrap $V(s_{t+n})$

λ - return : Weighted Avg over all n :

$G_t^\lambda = (1-\lambda) \sum \lambda^{n-1} G_t^n$.

Ex → Horizon = 15 , Batch = 1

Rewards - $r_0 \ldots\ldots r_{14}$

Values - $V_0 \qquad V_{14}$

V_{15} (Bootstrap).

$\gamma = 0.99, \lambda = 0.95$

For last step. $t = T-1$

$R^{\lambda 14}$ $r_t + \gamma V_{t+1}$ → $r_{14} + \gamma V_{15}$

$R^{\lambda 13}$. $r_{13} + \gamma((1-\lambda) V_{14} + \lambda R^{\lambda 14})$.

2 Losses (V2)

Actor = $-\left[\log \pi(a) \cdot sg\left(\underbrace{R^\lambda - V(s)}_{\text{Adv}}\right) + \underbrace{\eta \cdot H(\pi)}_{\text{Entropy}}\right]$

Critic = $\left(V(s) - sg\left(R^\lambda\right)\right)^2$

$V(s)$ → Value fn by the critic

R^λ → Returns

Target Critic → Critic both is trained & defines the target (V_λ bootstraps off it) → Chases itself

Fix = Slow EMA shadow gives the bootstrap value.

$Q\text{-target} = \tau \cdot Q\text{-target} + (1 - \tau) \cdot Q_\xi$

(τ → slow. window = $[1/1-\tau]$

Never Backprops.

Updated after Critic Step.

Training - inner loop (imagination) →

① seed real state → obs_step (posterior)

② Roll prior H steps → a = actor.sample(),

his = imagine_step, r, γ, v from heads

③ λ-returns backward (target critic Bootstraps)

④ Actor Loss → Actor only

⑤ Critic Loss → Critic only.

⑥ Optimizer steps

⑦ EMA target Update.